Оптимизация и численные методы

Болонкин Максим Викторович

Филиал МГУ им. М.В. Ломоносова в г. Ташкенте

Generally

- This is an overview course, there is no single story line
- The course objective: to introduce you to some of the most relevant computational and numerical tools commonly used in research and engineering
- The course materials are borrowed from various sources, I put attribution wherever possible
- Slides and course materials are predominantly in English
- This course has been offered since 2018, I used to change the actual topic several times
- This time the course will return to the original content but with certain updates and improvements

Course outline

"Computational and numerical tools" (known as *"Optimization and numerical methods"* in your curriculum)

What this course is about:

- Some of the most relevant computational and numerical tools commonly used in research and engineering
- Tools = algorithms and implementations

What this course is **not** about:

- Minimal math component: some math for understanding, rare derivations, almost no analysis/proofs
- Not a programming course: you are expected to know basic programming
- Breadth, not depth

Logistics

This course consists of **lectures and programming seminars**.

Lecture notes and video links will be posted online:
`https://www.mbolonkin.info/teaching/cs170-2026`

Seminars: you (students) will be teaching in the practical sessions
Final Project: details will be announced later

Final Grade: will be based on your performance in the seminars and the final project plus optional final exam if you want to improve your grade.

Final Project

The main source of your grade in this course will be your final project.

- One project per person (exceptions for approved collaborations)
- Project phases:
  - Proposal and approval
  - Midterm milestone
  - Final report and presentation
- The project will involve solving a practical problem using computational and numerical tools covered in this course.
- You will present your proposal and results in a final presentation.
- You are allowed to work on your thesis project as your final project for this course, but you need to get approval from me first.
- Final report must be submitted for you to receive your final grade.
- You are welcome to use AI tools as long as you understand the code and can explain it in your final presentation.

Programming language

Expected programming tools for this course:

- Matlab/Octave
- Python + Jupyter Notebooks (Recommended)

Why Python?

Python has well supported scientific libraries

- NumPy - supports wide variety of matrix operations
- SciPy - a lot of computational methods implemented
- MatPlotLib - library for plotting and visualizing
- Scikit-Image, Scikit-Learn, etc.
- Very good documentation

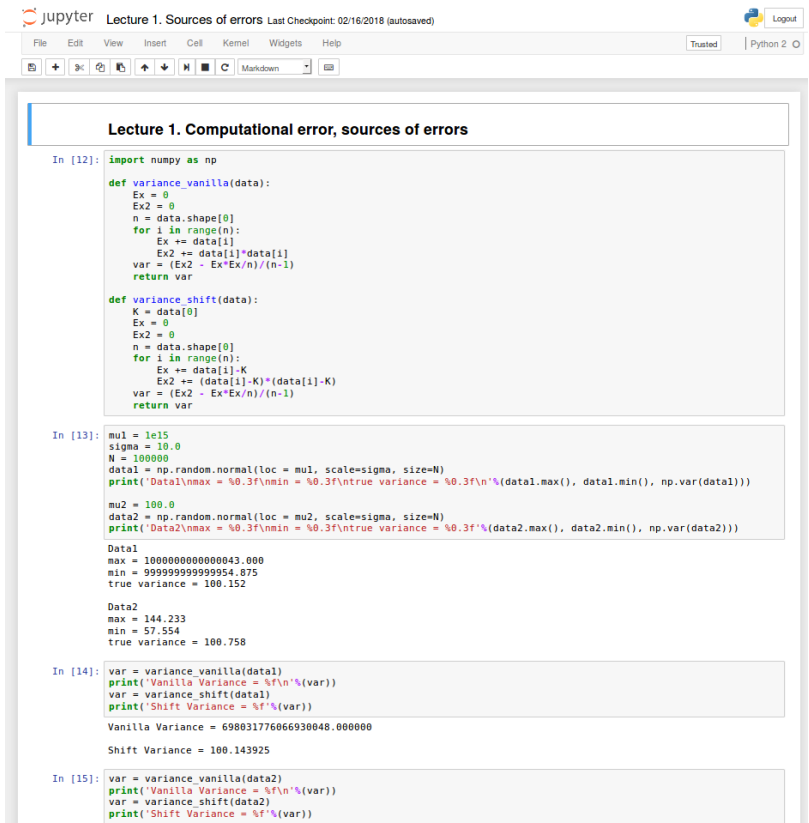
Why Python?

Python has well supported scientific libraries

- NumPy - supports wide variety of matrix operations
- SciPy - a lot of computational methods implemented
- MatPlotLib - library for plotting and visualizing
- Scikit-Image, Scikit-Learn, etc.
- Very good documentation

Jupyter Notebooks:

- Convenient tool for coding and debugging
- Good for reproducible research
- Getting more and more popular



Any questions?

Why bother?

- Computation allows exploring models without analytical solution
- This is usually the case for real-world problems
- Advances in hardware and software make it easier to use computational models

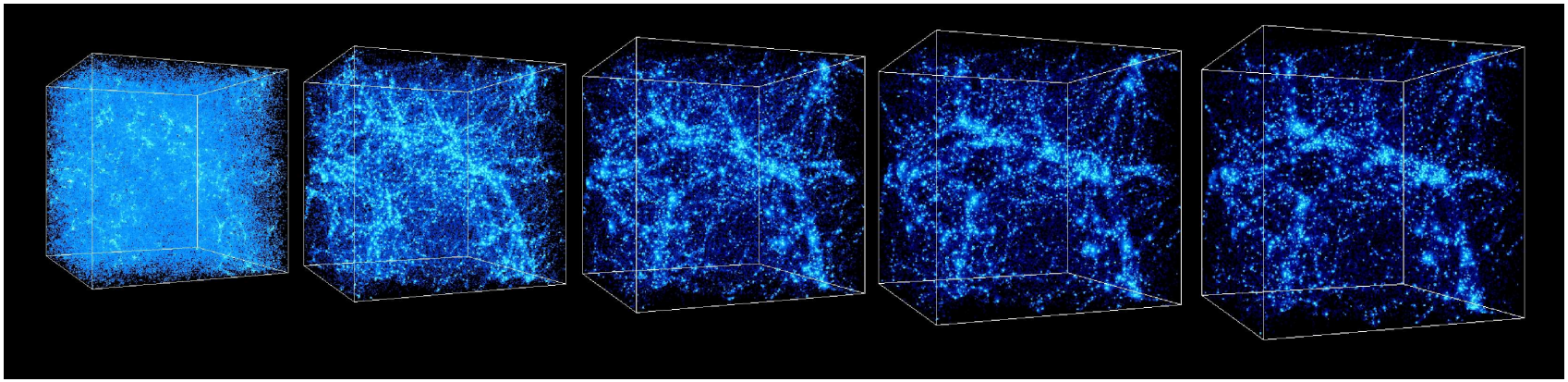


Рис.: Galaxy formation (cosmicweb.uchicago.edu)

Why bother?

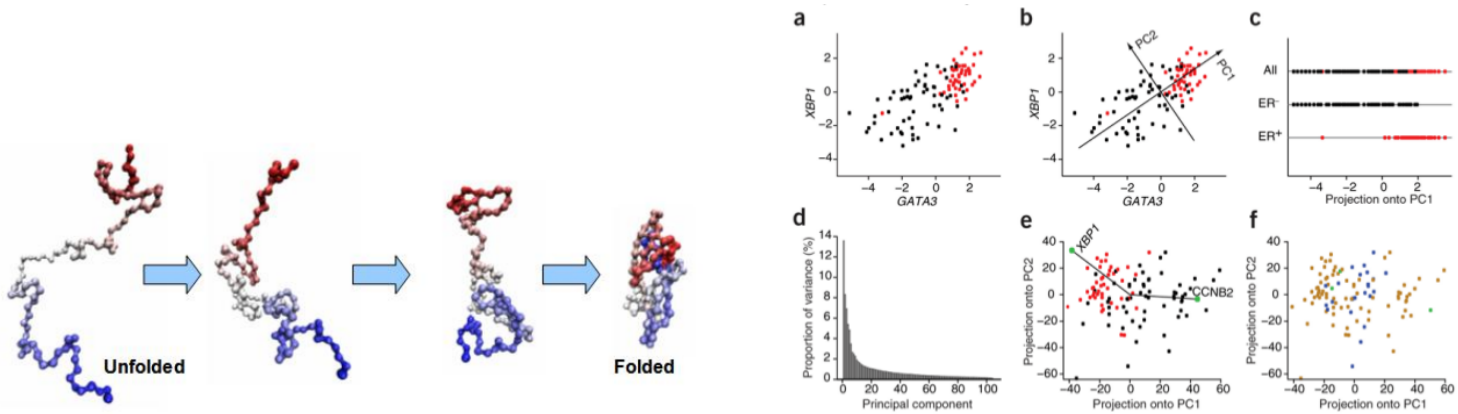


Рис.: Protein unfolding simulation and analysis of gene expression

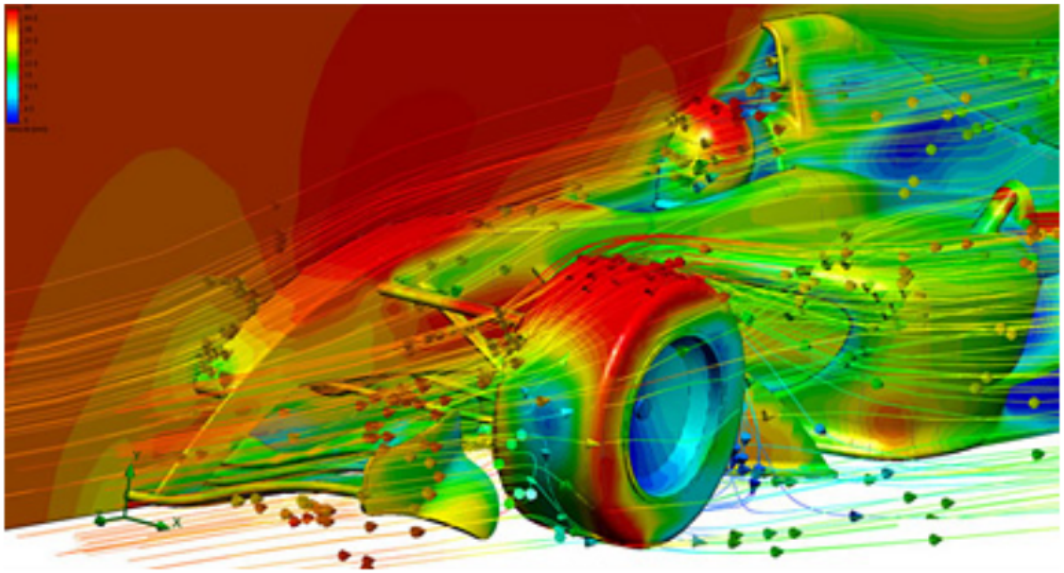


Рис.: Fluid dynamics simulation

In 2000 January-February issue of *Computing in Science and Engineering* editors selected 10 algorithms that influenced science and engineering in 20th century.

1. **Metropolis algorithm for Monte Carlo**

   1946: John von Neumann, Stan Ulam, and Nick Metropolis

   Through the use of random processes, this algorithm offers an efficient way to stumble toward answers to problems that are too complicated to solve exactly.

2. **Simplex method for linear programming**

   1947: George Dantzig, at the RAND Corporation

   An elegant solution to a common problem in planning and decision-making.

3. **Krylov subspace iteration methods**

   1950: Magnus Hestenes, Eduard Stiefel, and Cornelius Lanczos

   A technique for rapidly solving the linear equations that abound in scientific computation.

4. **The decompositional approach to matrix computations**

   1951: Alston Householder of Oak Ridge National Laboratory

   A suite of techniques for numerical linear algebra.

5 **The Fortran optimizing compiler**

1957: John Backus, IBM

Turns high-level code into efficient computer-readable code.

6 **QR algorithm for computing eigenvalues**

1959–61: J.G.F. Francis

Another crucial matrix operation made swift and practical.

7 **Quicksort algorithm for sorting**

1962: Tony Hoare

For the efficient handling of large databases.

8 **Fast Fourier transform**

1965: James Cooley of the IBM T.J. Watson Research Center and John Tukey of Princeton University and AT&T Bell Laboratories

Perhaps the most ubiquitous algorithm in use today, it breaks down waveforms (like sound) into periodic components.

9 **Integer relation detection**

1977: Helaman Ferguson and Rodney Forcade of Brigham Young University

A fast method for spotting simple equations satisfied by collections of seemingly unrelated numbers.

10 **Fast multipole method**

1987: Leslie Greengard and Vladimir Rokhlin of Yale University

A breakthrough in dealing with the complexity of n-body calculations, applied in problems ranging from celestial mechanics to protein folding.

Top 10 Algorithms of 21th Century

In 2016 editors of the CiSE updated the list of top 10 algorithms, adding the following:

1. **Newton and quasi-Newton methods**
2. **Jpeg**
3. **PageRank**
4. **Kalman filter**

Syllabus

What we are going to cover in this course:

- **Data fitting**
  Some topics: linear least squares, non-linear least squares, maximum likelihood, maximum a posteriori, expectation maximization
- **Optimization**
  Some topics: gradient-free optimization, gradient descent, non-linear optimization, heuristics
- **Dimensionaity reduction and component analysis**
  Some topics: PCA, ICA, t-SNE, LDA
- **Signal Processing**
  Some topics: signals, filters, DFT
- **Randomized methods**
  Some topics: random number sampling, Monte-Carlo methods, RANSAC
- **Differential equations**
  Some topics: numerical solutions for Ordinary Differential Equations (ODE), Partial Differential Equations (PDE)

General strategy for Numerical Solution

- Replacing difficult problem by simplified problem with the same or close enough solution

  Infinite $\rightarrow$ Finite
  Differential $\rightarrow$ Algebraic
  Continuous $\rightarrow$ Discrete
  Non-linear $\rightarrow$ Linear
  ...

- Solution obtained only approximates the exact solution of the original problem
  - Approximation before computation (empirical measurements, previous computation)
  - Approximation during computation (covered in next part)
  - Final result accuracy reflects all those errors, possibly amplified by an agorithm/problem